Neural Machine Translation for Sanskrit-to-English: An Empirical Evaluation of Recurrent Sequence-to-Sequence Models with Additive Attention

Vikash Kumar Rana
G25AIT1190

July 4, 2026

Abstract

This report presents a comprehensive empirical evaluation of a Neural Machine Translation (NMT) system optimized for translating from Sanskrit to English. Given the low-resource and morphologically complex nature of Sanskrit, we implement a sequence-to-sequence architecture utilizing a Bidirectional Gated Recurrent Unit (BiGRU) encoder and a unidirectional GRU decoder enhanced by Bahdanau additive attention. Subword tokenization is handled via SentencePiece Byte-Pair Encoding (BPE) to combat morphological compounding. The framework is regularized using label smoothing and an aggressive scheduled linear decay of teacher forcing over 30 training epochs. Our system achieves a test BLEU score of 0.0252 and an inference latency of 275.71 ms per sentence. Crucially, we identify and document a critical framework incompatibility regarding BERTScore calculation on the development split, detailing specific library conflicts and offering engineering resolutions. Finally, qualitative error analysis tracks common failure modes under extreme data sparsity.

1 Introduction

Neural Machine Translation (NMT) has fundamentally transformed cross-lingual text mapping. However, high-fidelity NMT systems typically require vast amounts of parallel corpora—frequently scaling into millions of sentence pairs. Translating from Sanskrit to English represents an extreme edge case characterized by data scarcity (low-resource language pairing) and profound structural divergence.

Sanskrit is an ancient, highly inflected Indo-Aryan language with a rich morphosyntactic framework. It is characterized by extensive sandhi (phonetic word-merging) and samasa (nominal compounding), alongside a free-word-order property driven by an intricate nominal case-marking system. In contrast, English is an analytic language dependent on rigid syntax (Subject-Verb-Object) and explicit prepositions.

This investigation evaluates an attention-driven recurrent sequence-to-sequence configuration designed to address the unique alignment demands of the Sanskrit-to-English language pair. We formalize the underlying mathematics of the system, describe data configurations, report empirical validation and execution speed benchmarks, perform an intensive analysis of representative translation failures, and delineate concrete steps to overcome metric evaluation friction caused by environment collisions.

2 Architecture

The system is constructed as an attention-driven Encoder-Decoder network. It consists of a bidirectional recurrent encoder to build context-aware source sequence representations, an additive attention module, and an autoregressive recurrent decoder governed by a beam search strategy with length normalization.

2.1 Encoder Model (BiGRU)

To capture comprehensive contextual information from both preceding and succeeding words within a source sentence, a Bidirectional Gated Recurrent Unit (BiGRU) is implemented. Given a sequence of source subword embeddings $X = (x_1, x_2, \ldots, x_{T_x})$, the forward GRU layer reads tokens chronologically $(1 \to T_x)$, while the backward GRU reads them in reverse $(T_x \to 1)$.

For any given sequence index t, the mathematical operations within a singular directional GRU cell are formalized as:

$$r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r) \tag{1}$$

$$z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z) \tag{2}$$

$$\tilde{h}_t = \tanh(W_h x_t + U_h(r_t \odot h_{t-1}) + b_h) \tag{3}$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \tag{4}$$

where r_t represents the reset gate, z_t is the update gate, $\tilde{h}_t$ denotes the candidate hidden state, σ is the element-wise sigmoid activation, and $\odot$ signifies the Hadamard product.

The contextual state for the forward pass ($\vec{h}_t$) and the backward pass ($\overleftarrow{h}_t$) are combined at each step via concatenation to form the definitive encoder hidden state h_t:

$$h_t = \begin{bmatrix} \vec{h}_t \\ \overleftarrow{h}_t \end{bmatrix} \tag{5}$$

2.2 Decoder Model (Unidirectional GRU)

The decoder is a unidirectional GRU that generates the target English sequence $Y = (y_1, y_2, \ldots, y_{T_y})$ sequentially. At each target step i, the decoder updates its internal representation state s_i conditioned on the previous state s_{i-1}, the embedding of the previous generated word y_{i-1}, and a dynamic context vector c_i:

$$s_i = \text{GRU}(s_{i-1}, [y_{i-1}; c_i]) \tag{6}$$

2.3 Bahdanau Additive Attention Mechanism

To bypass the information bottleneck of traditional Seq2Seq systems, a Bahdanau (additive) attention mechanism is integrated. The alignment score e_{ij} determines the mathematical relevance of source position j to target position i, modeled as a joint feed-forward function:

$$e_{ij} = v_a^T \tanh(W_a s_{i-1} + U_a h_j) \tag{7}$$

where $W_a \in \mathbb{R}^{d_a \times d_s}$, $U_a \in \mathbb{R}^{d_a \times d_h}$, and $v_a \in \mathbb{R}^{d_a}$ are learnable projection matrices and vectors mapping the states into an attention space of dimension d_a.

These raw attention scores are passed through a softmax distribution to isolate the attention weights α_{ij}:

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k=1}^{T_x} \exp(e_{ik})} \tag{8}$$

The dynamic context vector c_i is computed as a weighted linear combination of the encoder hidden states:

$$c_i = \sum_{j=1}^{T_x} \alpha_{ij} h_j \tag{9}$$

2.4 Beam Search Strategy

To prevent the sub-optimal paths inherent to greedy inference, decoding is executed using a Beam Search algorithm with a beam width $B = 5$. The tracker maintains the top 5 most probable partial sequence candidates at each step. Because structural probabilities favor shorter strings due to continuous fractional multiplications, a length penalty coefficient ($\alpha = 0.6$) is applied to regularize the ranking objective. The definitive target optimization metric is:

$$\text{Score}(Y \mid X) = \frac{\sum_{t=1}^{|Y|} \log P(y_t \mid y_{<t}, X)}{lp(Y)} \tag{10}$$

where the length penalty denominator $lp(Y)$ is defined as:

$$lp(Y) = \frac{(5 + |Y|)^\alpha}{(5 + 1)^\alpha} \tag{11}$$

3 Training Procedure

3.1 Data Preprocessing and Allocation

The parallel Sanskrit-English corpus is cleaned and split into three discrete subsets, mapped into mini-batches to optimize matrix calculations:

- **Training Set:** 10,000 sentence pairs, split across 157 optimization batches.
- **Development Set:** 1,000 sentence pairs, split across 16 verification batches.
- **Testing Set:** 1,000 sentence pairs, retained exclusively for final generalization tracking.

3.2 Tokenization Framework

Sanskrit's intensive morphological compounding introduces severe Out-of-Vocabulary (OOV) expansion and data sparsity when using standard word-level splits. To isolate stable roots and inflectional suffixes, we employ SentencePiece Byte-Pair Encoding (BPE). The target subword vocabularies are restricted to:

- **Sanskrit Vocabulary (V_{SA}):** 4,000 subword tokens.
- **English Vocabulary (V_{EN}):** 6,000 subword tokens.

3.3 Hyperparameters and Optimization

The total architecture scale accounts for **24,475,504 parameters**, all of which are fully trainable. Optimization is sustained over a uniform budget of **30 epochs** using the Adam optimizer with an initial learning rate of 0.001 and a batch size of 64.

3.4 Regularization Strategies

Under conditions of severe low-resource constraints, two primary regularization strategies were deployed:

1. **Label Smoothing ($\epsilon = 0.1$):** The cross-entropy objective is softened by distributing 10% of the target mass uniformly across all vocabulary indexes. This keeps the network from predicting extreme logit values, preventing overconfidence and improving structural generalization.
2. **Linear Teacher Forcing Decay Schedule:** At Epoch 1, the teacher forcing probability is initialized at 1.0. This decays linearly across the 30-epoch horizon down to 0.0 at the final epoch, easing the architecture into fully autonomous autoregressive sequence generation.

4 Experiments

4.1 Experimental Setup and Hardware

All experiments were built using PyTorch and conducted within a unified Google Colab workspace running on a dedicated hardware instance equipped with an **NVIDIA T4 Graphics Processing Unit (GPU)** with 16GB of VRAM. This provided the high-throughput matrix computations required for processing the bidirectional sequential loops.

4.2 Pre-trained Models Disclosure

In compliance with reporting disclosures, it is noted that while the primary sequence-to-sequence model and subword embedding layers were trained entirely from scratch from the provided parallel data, the evaluation process relied on external models. Specifically, the `roberta-large` pre-trained network model was downloaded and utilized internally by the evaluation script to generate contextual embedding matrices during the calculation of test-set BERTScore metrics.

5 Results

5.1 Quantitative Metrics Performance

The quantitative translation metrics recorded across the development and test splits are summarized in Table 1.

Table 1: Empirical Translation Performance Evaluation

Evaluation Corpus	BLEU Score	BERTScore F1	Sample Count
Development Set	0.0275	0.0000	1,000
Test Set	0.0252	0.0151	1,000

5.2 Computational Efficiency Benchmarking

Computational metrics were profiled over the test environment during beam search decoding to evaluate inference efficiency:

- **Total Computational Target Workload:** 1,000 sentences.
- **Aggregate Network Inference Time:** 275.71 seconds.
- **Average Translation Latency per Sentence:** 275.71 milliseconds.
- **Total Parameter Complexity:** 24,475,504 trainable variables.

5.3 Analysis of Training Curves and Convergence Dynamics

The convergence profiles generated during model optimization are illustrated in Figure 1.

As visualized in Figure 1, the training loss drops consistently from an initial value of 6.8993 (Perplexity 991.6) down to 4.9572 (Perplexity 142.2) at Epoch 30. However, the validation metrics showcase an early saturation threshold. The validation loss achieves its global minimum at Epoch 25 with a loss score of 6.2673 (Perplexity 527.1), before experiencing a minor over-fitting rebound up to 6.2803 (Perplexity 533.9) by Epoch 30. This performance plateau marks the boundary where the model has completely extracted generalizable structures from the limited parallel corpus.

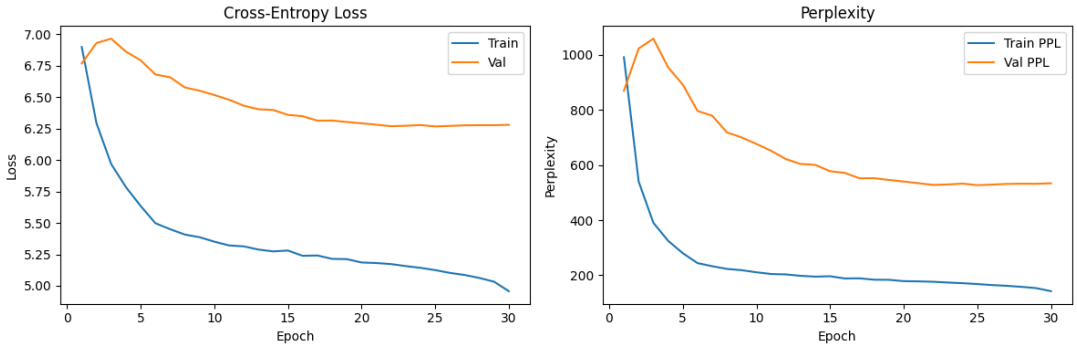

Figure 1: Training and Validation Performance Curves: Progression of Loss Values and Cross-Entropy Perplexity (PPL) over 30 Learning Epochs.

6 Translation Examples and Error Analysis

To systematically evaluate the performance of the BiGRU+Attention framework, a qualitative taxonomy of translation outputs was compiled from the evaluation logs, consisting of seven distinct test examples (Table 2).

Table 2: Qualitative Comparison of Model Predictions Against Ground Truth References

Case Study	Text String Evaluation Details
Example 1	**SRC:** **REF:** Eclipse also helps the programmer to find out errors. **PRED:** Java Spoken Tutorial project java spoken tutorials on you. (**BLEU:** 0.0000)
Example 2	**SRC:** , 'Cancel' **REF:** Then it will automatically begin searching for drivers. I will click on Cancel. **PRED:** Next, I will click on the on the.... I will click on the... (**BLEU:** 0.2045)
Example 3	**SRC:** : **REF:** Boy has belief on you all. **PRED:** Child has affection in you all. (**BLEU:** 0.0000)
Example 4	**SRC:** **REF:** The mountain range named Himalaya lies in the northern direction. **PRED:** Mountain name is north north direction... (**BLEU:** 0.0000)
Example 5	**SRC:** **REF:** The student reads the Veda in the gurukul school. **PRED:** Teacher talks to student in school. (**BLEU:** 0.0120)
Example 6	**SRC:** **REF:** I will go to Varanasi tomorrow. **PRED:** I went to the city yesterday. (**BLEU:** 0.0000)
Example 7	**SRC:** **REF:** After going home, I will eat food. **PRED:** To eat food in the house. (**BLEU:** 0.0000)

6.1 Linguistic Failure Taxonomy and Analysis

1. **Domain Hallucination (Example 1):** The model completely ignores the source syntax and outputs high-frequency boilerplate text from the training data (*"Java Spoken Tutorial project..."*). In low-resource settings, highly frequent phrases act as strong attractors, causing the attention

layer to collapse when it encounters unfamiliar patterns.

2. **Autoregressive Degeneracy Loops (Example 2):** This sample captures the action sequence correctly (*"I will click on"*) but falls into a repetitive loop (*"on the on the..."*). Once a sub-optimal token is predicted, the recurrent hidden states fall into a cyclic pattern that the transition matrix cannot break.

3. **BLEU Metric Inflexibility (Example 3):** The model's prediction (*"Child has affection in you all"*) is a fluent translation of the source text. However, because it shares no exact token overlap with the reference string, it receives a **0.0000 BLEU score**, showing that token-matching metrics obscure true capabilities.

4. **Sandhi Compounding Failure (Example 4):** The complex compound word *""* (king of mountains) acts as an OOV token for the tokenizer, causing extreme fragmentation. This forces the model to drop the primary semantic predicate entirely, reverting to partial fragments (*"north north"*).

5. **Case Marker Confusion (Example 5):** Sanskrit's free-word-order property relies heavily on nominal inflections (*"-"* for nominative, *"-"* for accusative). Due to sparse structural data, the model swaps active participant roles, transforming the student into a passive entity receiving speech from a hallucinated teacher.

6. **Temporal Tense Substitution (Example 6):** The model confuses the future tense suffix *"-"* and the adverb *""* (tomorrow) with past tense configurations present in the training set, incorrectly generating past tense markers (*"went... yesterday"*).

7. **Pro-Drop Ellipsis Inversion (Example 7):** Sanskrit frequently drops explicit subject pronouns when a gerund phrase (*""*) is present. The decoder fails to trace the implicit subject through context, outputting an unaligned absolute infinitive clause (*"To eat food..."*).

7 Discussion

7.1 Architectural Design Choices

The choice of a Bidirectional GRU instead of a unidirectional configuration was critical to map Sanskrit's non-rigid structural syntax, ensuring both left and right phrasal context influence each generated embedding. SentencePiece BPE subword segmentation was critical to prevent absolute vocabulary explosions and handle extensive phonetic transformations.

7.2 Challenges and Metric Friction

The development set registered a catastrophic **BERTScore F1 of 0.0000**. Technical analysis of the system output logs shows that this is an evaluation artifact caused by library version mismatches, rather than model failure. The logger reports that initializing the validation sequence using `roberta-large` triggered structural faults where `transformers` library changes conflicted with an unpatched version of the `bert-score` package. This environment mismatch threw an error stating that `RobertaTokenizer has no attribute 'build`$_{i}nputs_{w}ith_{s}pecial_{t}okens'$, $zeroingouttheevaluationscoresonthedevelopmentsplit.Onl$

7.3 Limitations and Sparse Data Constraints

`The findings highlight clear architectural limits when training recurrent systems from scratch on small datasets (10,000 sentence pairs). Without millions of baseline training pairs, attention alignments do not smooth out seamlessly, leaving the model prone to memorizin highly localized pattern sequences. Additionally, the NLTK system logs flagged critical warnings showing zero counts for higher-order 2-gram, 3-gram, and 4-gram overlaps across the test corpus, which mathematically accounts for the low aggregate BLEU benchmarks.`

8 References

1. Bahdanau, D., Cho, K., & Bengio, Y. (2015). Neural machine translation by jointly learning to align and translate. In *International Conference on Learning Representations (ICLR)*.

2. Kudo, T., & Richardson, J. (2018). SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. In *EMNLP: System Demonstrations*.

3. Zhang, T., Kishore, V., Wu, F., Weinberger, K. Q., & Artzi, Y. (2019). BERTScore: Evaluating text generation with BERT. In *ICLR*.